

APRENDIZADO PROFUNDO • PONTO DE PARTIDA

Redes Neurais **Artificiais**

Do neurônio único à rede que aprende. O mesmo problema de classificação que já resolvemos — agora com um modelo inspirado no cérebro.



Uma rede neural é, no fundo, **muitos modelos simples** empilhados — e hoje vamos abrir essa caixa.

ONDE ESTAMOS

A rede neural é mais um modelo supervisionado

Já treinamos modelos que aprendem com **exemplos rotulados** para prever uma classe. A rede neural entra na mesma família.

Regressão Logística

fronteira linear, simples e interpretável.

Árvore de Decisão

sequência de perguntas — regras claras.

Random Forest

um conjunto de árvores, mais robusto.

Rede Neural (hoje)

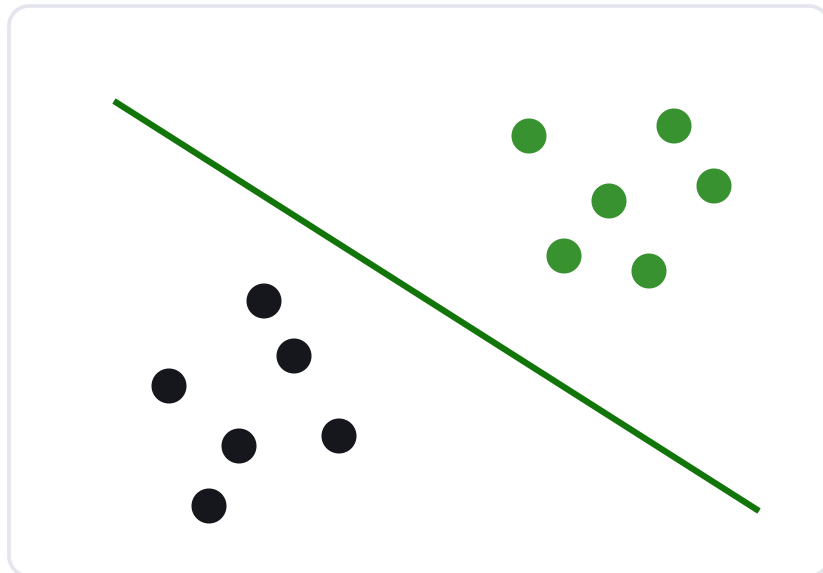
combina muitos neurônios em camadas.

A pergunta não muda: dado um conjunto de variáveis, a que **classe** este exemplo pertence? Muda a **forma** de aprender a resposta.

MOTIVAÇÃO

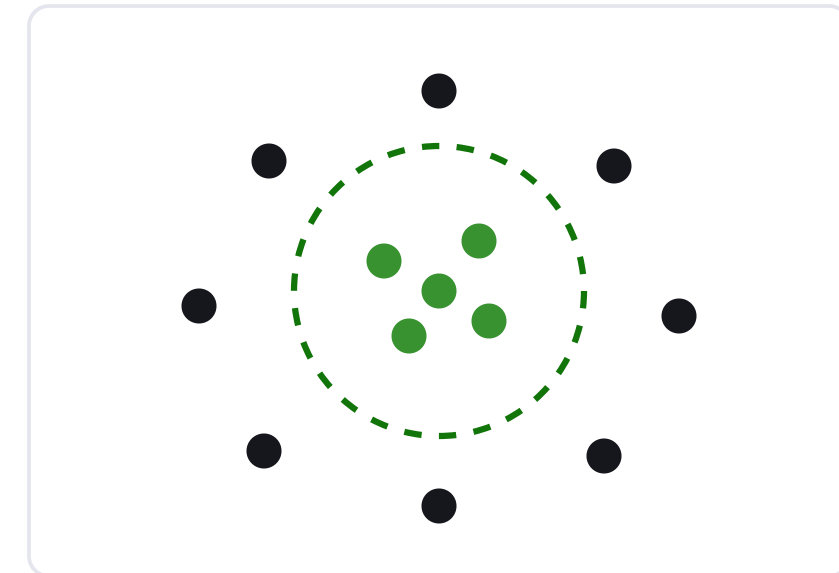
O que uma reta não consegue separar

• Linearmente separável



Uma **reta** já separa as classes — a **regressão logística** resolve.

• Não linear



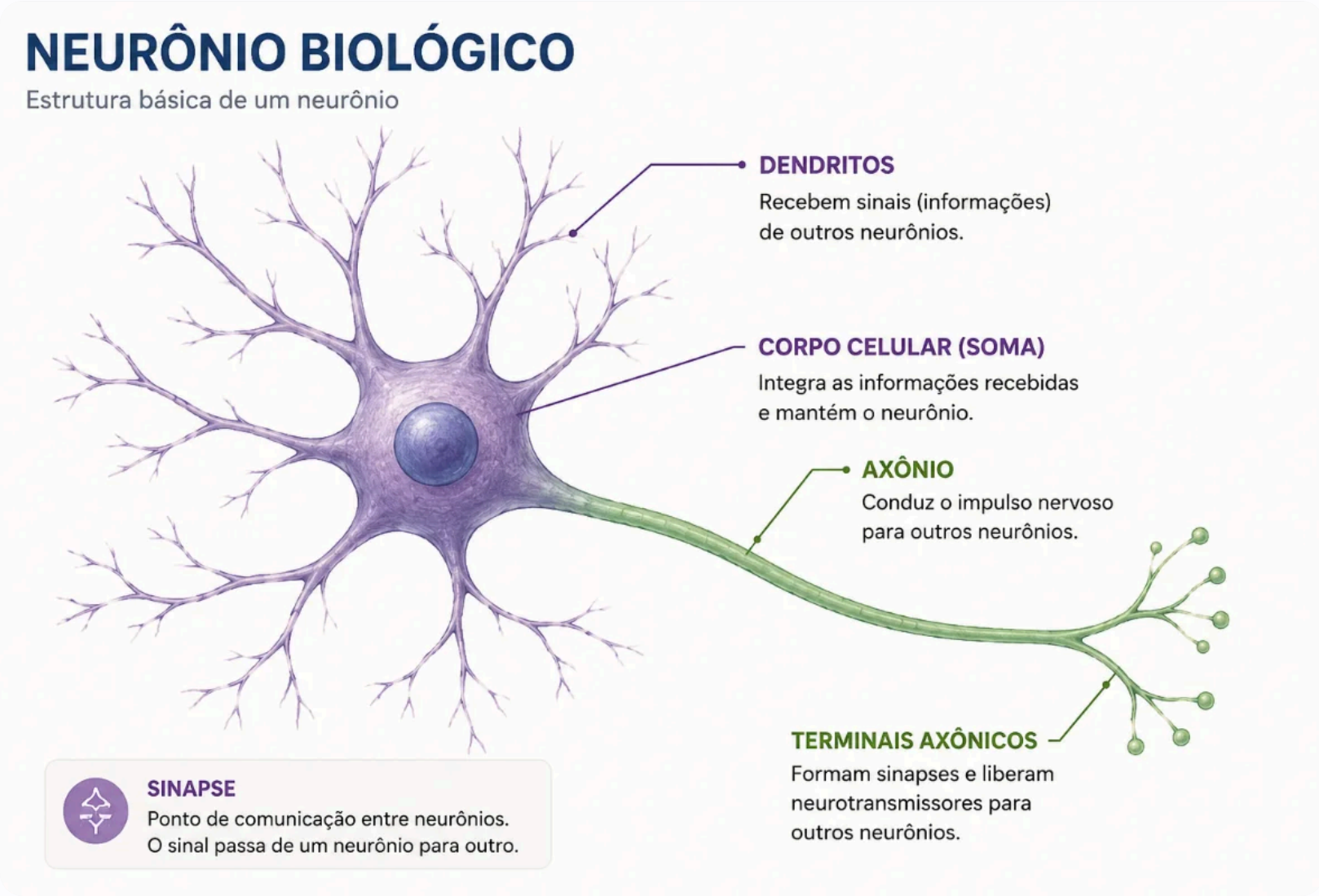
Nenhuma reta dá conta — a **rede aprende a curva** a partir dos dados.

A ideia central: em vez de escolher a forma da fronteira à mão, deixamos a rede **descobri-la a partir dos dados**.

INSPIRAÇÃO BIOLÓGICA

Do neurônio biológico ao artificial

• Neurônio biológico • foto real



Cada parte do neurônio tem um **equivalente** no modelo — na mesma ordem do sinal.

NO NEURÔNIO

NO MODELO ARTIFICIAL

Dendritos

recebem os sinais → as **entradas** ($x_1, x_2, \dots$)

Corpo celular (soma)

integra os sinais → **soma ponderada + viés**

Axônio

conduz o resultado → a **função de ativação**

Terminais axônicos

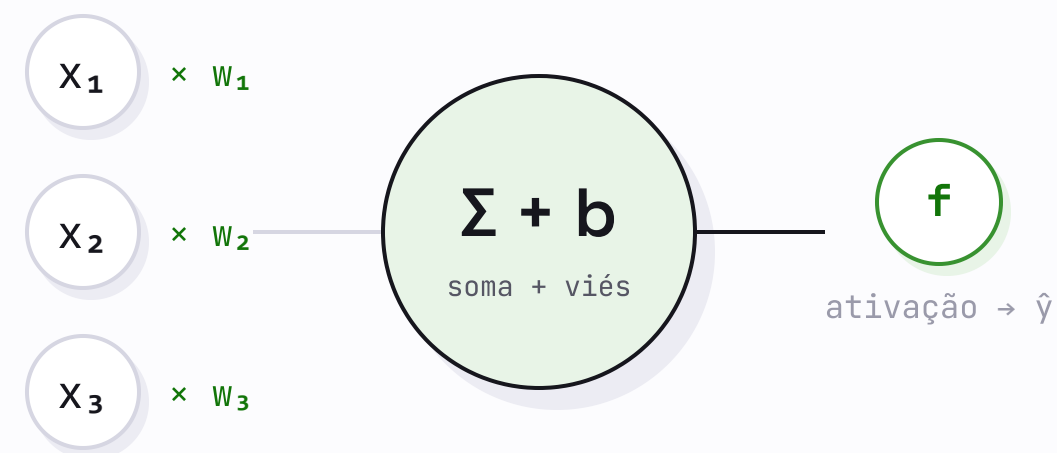
transmitem adiante → a **saída $\hat{y}$**

dendritos → corpo celular (soma) → axônio → terminais

Atenção: é só uma analogia. O neurônio artificial é uma **fórmula matemática simples** — não uma célula.

ANATOMIA

As partes de um neurônio artificial



A FÓRMULA, EM PALAVRAS

Multiplique cada **entrada** pelo seu **peso**, some tudo, adicione o **viés** e passe o resultado por uma **função de ativação**. Só isso.

CÁLCULO NA MÃO

Um neurônio decidindo, passo a passo

Decidir se vale a pena **levar guarda-chuva**, a partir de duas variáveis:

ENTRADAS	chance de chuva $x_1 = 0,8$ · vai sair a pé $x_2 = 1$
PESOS	$w_1 = 3$ · $w_2 = 1$
VIÉS	$b = -2$

```
# 1 · soma ponderada + viés
```

```
z = (0.8×3) + (1×1) + (-2)
```

```
z = 1.4
```

```
# 2 · ativação (degrau): z > 0 ?
```

```
 $\hat{y} = 1$  # sim → leve o guarda-chuva
```

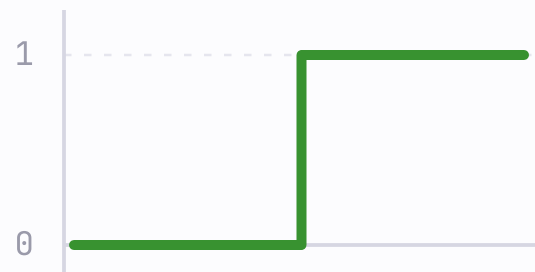
Treinar a rede é, no fundo, **descobrir bons pesos e viés** — não definimos esses números à mão.

A CHAVE DA NÃO LINEARIDADE

Funções de ativação

É a ativação que dá à rede o poder de aprender padrões **não lineares**. Sem ela, mil camadas viram uma só reta.

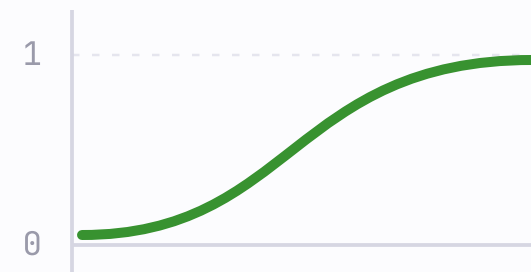
Degrau (step)



Devolve **0 ou 1**: o neurônio dispara ou não — um corte seco no zero.

uso • o perceptron original

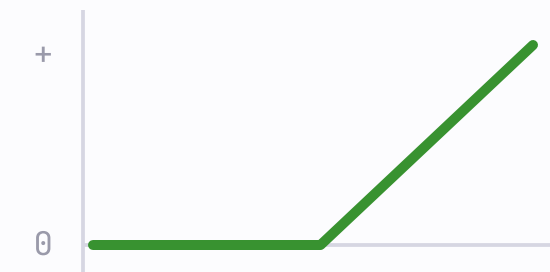
Sigmoide (sigmoid)



Comprime tudo entre **0 e 1**, suavemente — vira uma **probabilidade**.

uso • saída binária

ReLU



Zera o que é **negativo** e mantém o **positivo** — simples e rápida.

uso • camadas ocultas

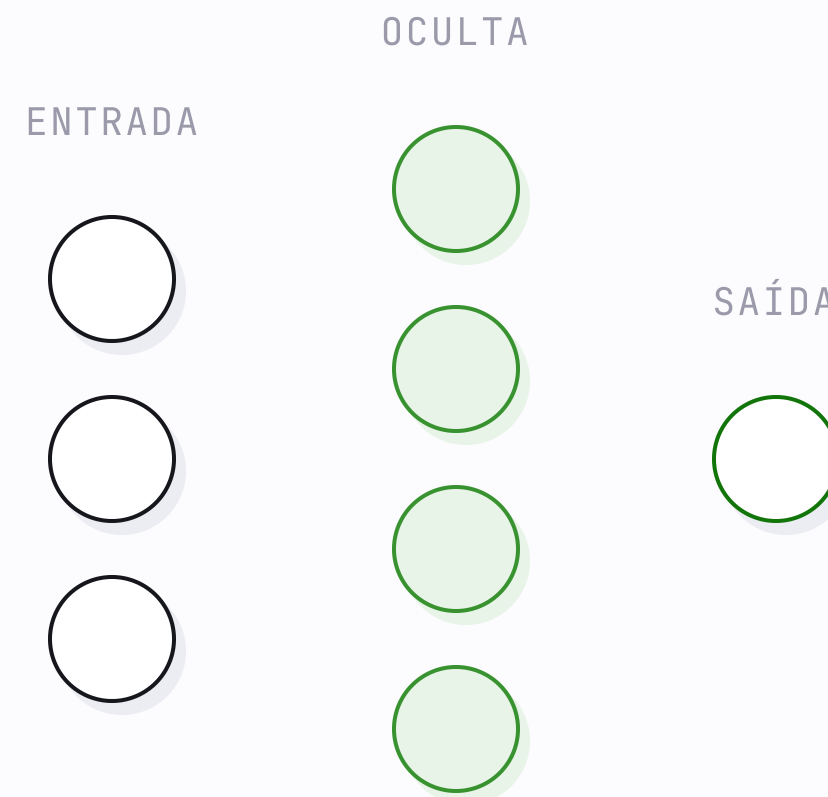
No scikit-learn, a **ReLU** é a ativação padrão das camadas ocultas do **MLPClassifier**.

PARTE 2 • 0 APRENDIZADO

Como a rede aprende

De um neurônio a uma rede inteira — e o
ciclo que ajusta os pesos até errar menos.

De um neurônio a uma rede em camadas



Camada de entrada

uma posição por variável do exemplo.

Camadas ocultas

onde os padrões são combinados e aprendidos.

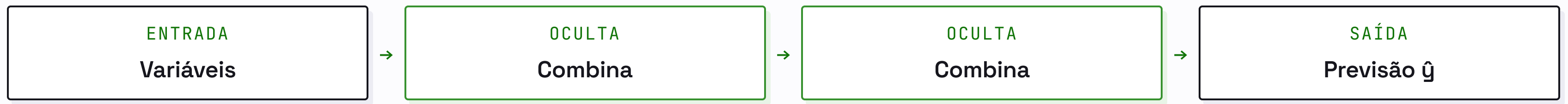
Camada de saída

a previsão final — aqui, a classe.

PASSO 1 • PREVER

Propagação direta (forward)

O dado entra de um lado e atravessa a rede, camada por camada, até virar uma **previsão**.



Cada neurônio faz aquele cálculo simples do início — **soma ponderada + viés + ativação**. A rede só repete isso muitas vezes.

O EXEMPLO CLÁSSICO

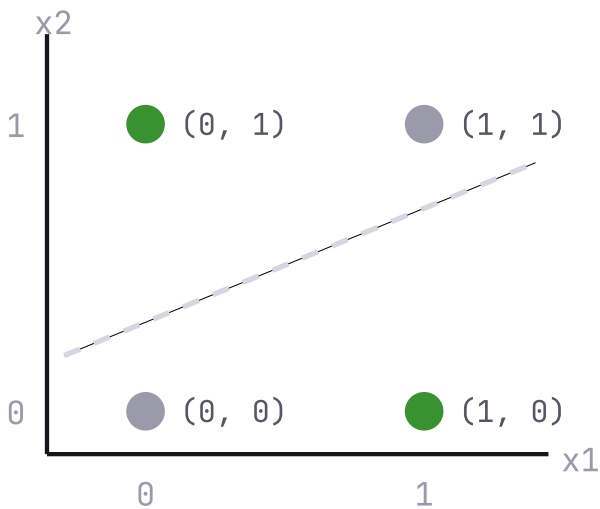
Por que precisamos de camadas ocultas

O problema **XOR**: a saída é 1 quando as entradas são **diferentes**.

Tabela XOR

x1	x2	SAÍDA
0	0	0
0	1	1
1	0	1
1	1	0

Distribuição dos pontos



As classes ficam em **diagonais opostas**.

- Nenhuma **reta única** separa esses quatro pontos.
- Um modelo **linear simples** falha aqui.

💡 **Ideia-chave:** o XOR mostra que nem todo problema pode ser resolvido com uma única fronteira linear.

A SOLUÇÃO INTUITIVA

O que a camada oculta faz

Ela cria **combinações intermediárias** antes da decisão final.

Entradas

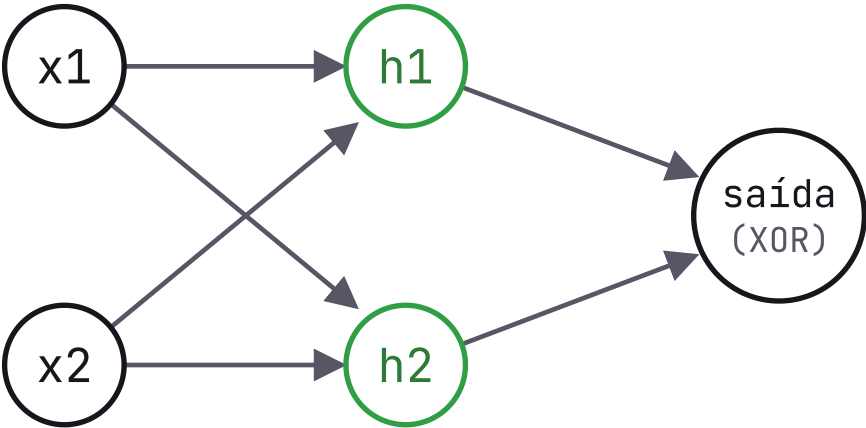
x1

x2

Problema: não há uma reta única que separe as classes.

0 ●	1 ●
1 ●	0 ●

Camada oculta



h1 aprende uma fronteira simples
h2 aprende outra fronteira simples

- ⓘ A camada oculta cria **representações intermediárias**.
- 🔄 A saída final **combina** essas representações.

💡 **Ideia-chave:** combinar fronteiras simples permite formar uma decisão mais complexa.

PASSO 2 • MEDIR O ERRO

Errar menos — a função de custo

Para melhorar, a rede precisa de uma nota: **o quanto longe** a previsão ficou da resposta certa.

Previsão $\hat{y}$

o que a rede achou — ex.: "70% sobreviveu".

Resposta y

o rótulo verdadeiro do exemplo de treino.

Custo (erro)

um número que cresce quando $\hat{y}$ e y discordam.

Treinar é **minimizar esse custo**: encontrar os pesos que deixam a previsão o mais próxima possível da verdade, em todos os exemplos.

PASSO 3 • MELHORAR

Gradiente descendente — descer a montanha

Imagine o custo como o **relevo de um vale**, na neblina. Você quer chegar ao ponto mais baixo, mas só enxerga o chão sob seus pés.

- 01 Sinta a **inclinação** do terreno onde você está.
- 02 Dê um **passo morro abaixo**, na direção mais íngreme.
- 03 Repita. Aos poucos, você chega ao **fundo do vale**.



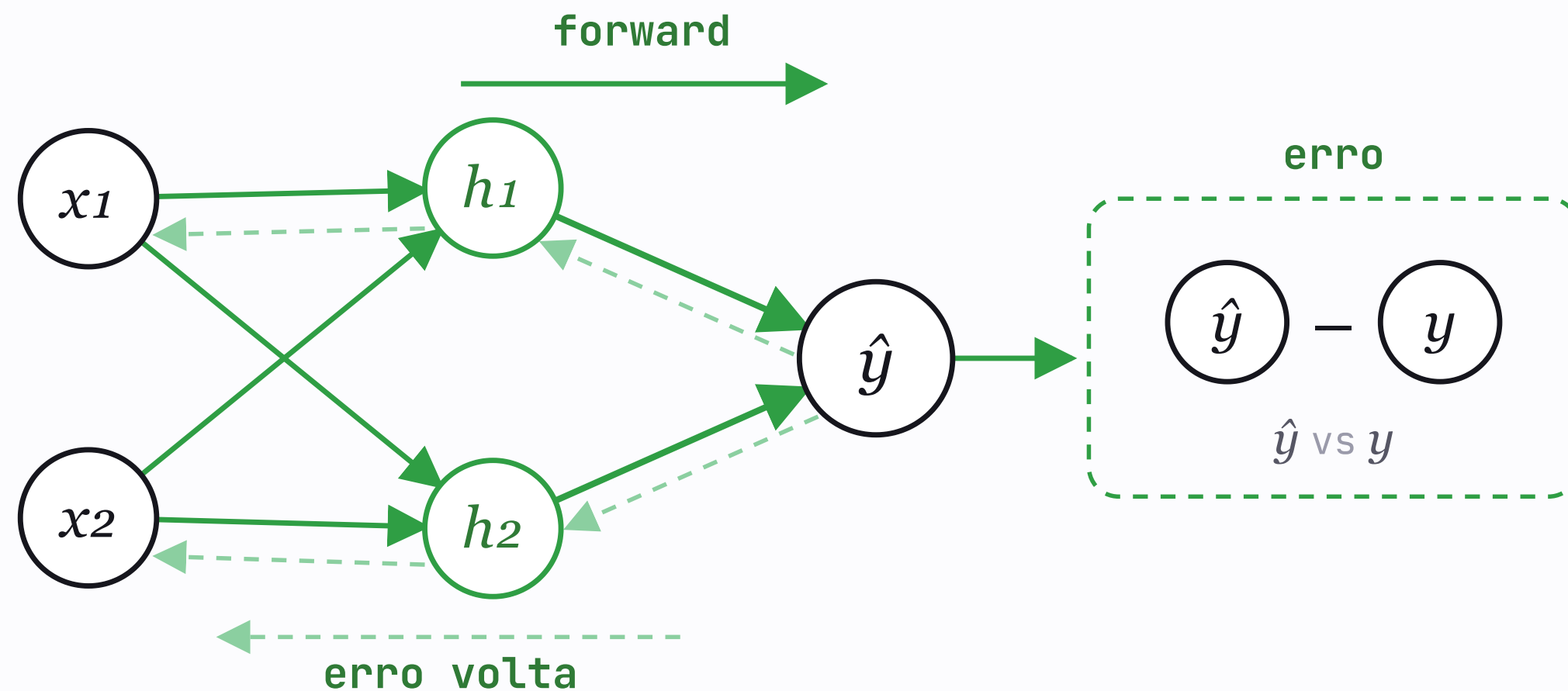
→ tamanho do passo = taxa de aprendizado

A cada passo, ajustamos os pesos para reduzir o erro.

O MOTOR DO TREINO

Retropropagação (backpropagation)

Como saber **qual peso** ajustar, e quanto? A rede distribui a "culpa" pelo erro de trás para frente.



Ideia-chave:

o **erro volta** pela rede para mostrar quais pesos contribuíram mais para o resultado errado.

1 · PREVÊ
Forward

2 · COMPARA
Mede o erro

3 · VOLTA
Backprop

4 · AJUSTA
Novos pesos

↻ Uma **época** = passar por todos os exemplos uma vez e repetir este ciclo. Treinar leva muitas épocas.

O QUE VOCÊ CONTROLA

Os hiperparâmetros de uma rede

HIPERPARÂMETRO	O QUE CONTROLA	NO SCIKIT-LEARN
Camadas e neurônios	o "tamanho" e a capacidade da rede.	<code>hidden_layer_sizes</code>
Taxa de aprendizado	o tamanho do passo morro abaixo.	<code>learning_rate_init</code>
Épocas	quantas vezes percorrer os dados.	<code>max_iter</code>
Ativação	a função das camadas ocultas.	<code>activation</code>

Hiperparâmetro é o que **nós** definimos antes de treinar. **Pesos** são o que a **rede** aprende durante o treino.

RECAPITULANDO • ATÉ AQUI

Da unidade ao aprendizado

01 Um **neurônio** é soma ponderada + viés + ativação.

02 Empilhados em **camadas**, eles aprendem padrões não lineares.

03 A rede prevê (**forward**), mede o **custo** e ajusta os pesos (**backprop + gradiente**).

Já temos a teoria. Hora de ver isso **rodando em código**.

● A seguir: mãos ao `scikit-learn`

PARTE 3 • NA PRÁTICA

Mãos ao código

A rede neural no mesmo pipeline que já usamos — scikit-learn, do treino à comparação.

Os mesmos dados, um novo modelo

Vamos usar os datasets que já conhecemos — ambos de **classificação binária**.

● Dataset · Titanic

PERGUNTA

o passageiro **sobreviveu**?

VARIÁVEIS

sexo, idade, classe, tarifa, família a bordo

ALVO

survived · 0 ou 1

● Dataset · Default of Credit Card

PERGUNTA

o cliente vai **dar calote** no mês seguinte?

VARIÁVEIS

limite, faturas, pagamentos, atrasos

ALVO

default · 0 ou 1

Já comparamos **Regressão Logística, Árvore e Random Forest** neles. Agora adicionamos a **rede neural** — e usamos a mesma régua para avaliar.

A rede entra no mesmo pipeline

```
from sklearn.neural_network import MLPClassifier

# a rede também é sensível à escala –
# reaproveitamos o pré-processador da Regressão Logística
mlp = MLPClassifier(
    hidden_layer_sizes=(16, 8), # 2 camadas ocultas
    activation="relu",
    max_iter=500,
    random_state=42,
)

mlp_model = Pipeline(steps=[
    ("preprocessor", preprocessor_reg), # scaler + one-hot
    ("classifier", mlp),
])
```

- ! Como a **Regressão Logística**, a rede precisa de variáveis **padronizadas** — por isso o **StandardScaler**.
- ✓ Mesmo **treino/teste** e mesmo **pré-processamento** dos outros modelos: comparação justa.
- ✓ (16, 8) = duas camadas ocultas, com 16 e 8 neurônios.

O FLUXO DE SEMPRE

Treinar, prever e avaliar

```
# treina exatamente como os outros modelos
mlp_model.fit(X_train, y_train)

# prevê no conjunto de teste
y_pred_mlp = mlp_model.predict(X_test)

# mesma função de avaliação do notebook
avaliar_modelo("Rede Neural (MLP)", y_test, y_pred_mlp)
```

A BOA NOTÍCIA

A interface é **idêntica** à dos outros modelos: `.fit()` e `.predict()`.

Trocar Random Forest por uma rede neural muda **duas linhas** — o resto do notebook continua igual. É essa consistência do scikit-learn que vamos explorar na atividade.

Quatro modelos, a mesma régua

MODELO	ACURÁCIA	PRECISÃO	RECALL	F1
Regressão Logística	0,785	0,732	0,698	0,714
Árvore de Decisão	0,726	0,640	0,663	0,651
Random Forest	0,803	0,850	0,593	0,699
Rede Neural (MLP)	?	?	?	?

A última linha é a sua tarefa. Treinaremos a MLP juntos e preencheremos esses números — depois discutimos: a rede venceu? Em qual métrica? Compensou a complexidade?

BOAS PRÁTICAS

Três cuidados ao usar uma rede

Padronizar os dados

redes são sensíveis à escala. Sem normalizar, o treino fica lento e instável.

Separar treino e teste

avale sempre em dados que a rede **nunca viu** — ou a nota engana.

Vigiar o overfitting

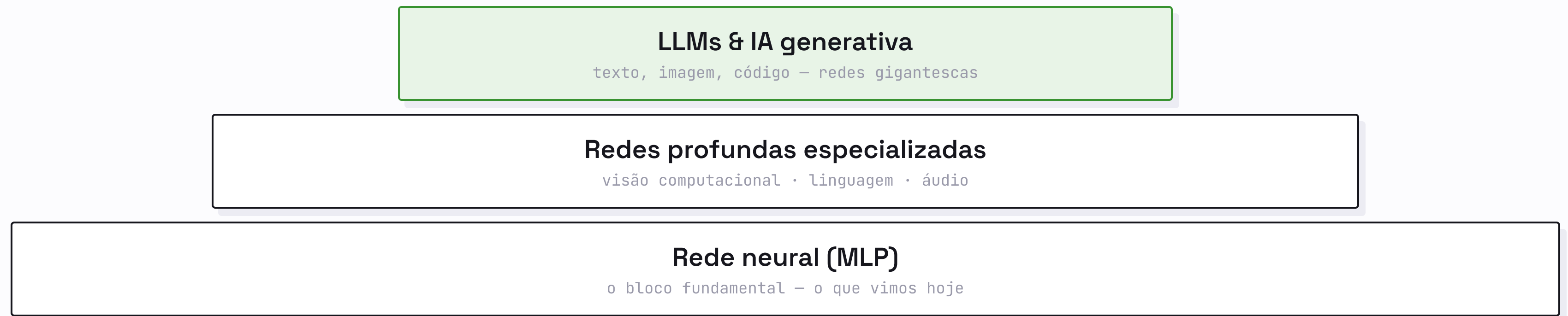
uma rede grande decora o treino. Se vai muito bem no treino e mal no teste, é sinal de alerta.

Mais neurônios e épocas **nem sempre é melhor**. Muitas vezes, um modelo simples e bem ajustado vence uma rede grande mal cuidada.

PARA ALÉM DA AULA

Da MLP ao aprendizado profundo

A rede de hoje é pequena. **Empilhar muitas camadas** é o que chamamos de **deep learning**.



Tudo isso parte do **mesmo neurônio** do início da aula. A diferença é **escala, dados e arquitetura** — não o princípio.

CRITÉRIO PRÁTICO

Quando escolher uma rede neural

Costuma valer a pena

- ✓ Dados **abundantes** e padrões **complexos**.
- ✓ Imagem, texto, áudio — **sinais brutos**.
- ✓ Quando a **performance** importa mais que explicar o porquê.

Talvez não compense

- Poucos dados, tabela pequena — **modelos simples** vão bem.
- Quando você precisa **explicar a decisão** com clareza.
- Para tabelas, **Random Forest** costuma ser um forte baseline.

Não existe modelo "melhor" universal. A escolha depende dos dados, do objetivo e do quanto você precisa interpretar o resultado.

PARTE 4 • FIXAÇÃO • ~50 MIN

Vamos fixar

Conceitos, um cálculo na mão e a mão na massa — estendendo o notebook que já conhecemos.

PARTE 1 • INDIVIDUAL

Verifique seu entendimento

• Questão 1 • múltipla escolha

O que a **função de ativação** acrescenta a um neurônio?

- A ☐ Mais velocidade ao treino
- B ☐ A capacidade de aprender padrões não lineares
- C ☐ A normalização das variáveis
- D ☐ A divisão entre treino e teste

• Questão 2 • verdadeiro ou falso

- (☐ V) (☐ F) Os pesos são definidos por nós antes do treino.
- (☐ V) (☐ F) Uma camada oculta permite resolver o problema XOU.
- (☐ V) (☐ F) A retropropagação ajusta os pesos para reduzir o erro.
- (☐ V) (☐ F) Mais neurônios sempre melhoram o desempenho no teste.

Justifique cada item falso reescrevendo-o de forma correta no caderno.

PARTE 2 · INDIVIDUAL

Calcule e associe

• Questão 3 · cálculo na mão

Um neurônio com **ativação degrau**. Calcule **z** e a saída **ŷ**:

ENTRADAS	$x_1 = 1 \cdot x_2 = 0 \cdot x_3 = 1$
PESOS	$w_1 = 0,5 \cdot w_2 = -1 \cdot w_3 = 2$
VIÉS	$b = -1$
PEDE-SE	$z = ? \cdot \hat{y} = ?$ (dispara se $z > 0$)

• Questão 4 · associação

Ligue cada termo à sua descrição:

A	Peso	• () mede o erro da previsão
B	Época	• () uma passada por todos os dados
C	Custo	• () onde os padrões são combinados
D	Camada oculta	• () a força de uma entrada, aprendida no treino

PARTE 3 • EM DUPLAS • NO NOTEBOOK

Coloque a rede na disputa

PONTO DE PARTIDA

O notebook que já compara **Regressão Logística, Árvore e Random Forest** no Titanic.

- 01 Adicione um **MLPClassifier** no mesmo pipeline (com o scaler).
- 02 Treine, preveja e **calcule as quatro métricas** com a função do notebook.
- 03 Preencha a **linha que faltava** na tabela comparativa.

• Desafios extras

- a Mude **hidden_layer_sizes** e veja o efeito no F1.
- b Repita tudo no dataset **Default of Credit Card**.
- c **Discussão:** a rede venceu a Random Forest? Valeu a complexidade?

Entrega: o notebook com a tabela completa e **3 a 4 linhas** de conclusão.

SÍNTESE

O que levamos desta aula

- 01 O **neurônio** é simples: soma ponderada + viés + ativação.
- 02 Em **camadas**, neurônios aprendem padrões que uma reta não captura.
- 03 A rede aprende por **forward** → **custo** → **backpropagation**, ajustando os pesos.
- 04 No **scikit-learn**, ela entra no mesmo pipeline — e disputa de igual para igual com os outros modelos.

PRÓXIMA AULA

Avaliação de modelos → validação cruzada e ajuste de hiperparâmetros